

CSC 330 Programming Languages

Program #2 – Easy Version

Inter Process Communication (on Mac/Linux)

Directions:

There are many ways that two different languages can communicate. Interprocess communication (or IPC for short) is one way of accomplishing this. In this assignment you will use IPC to allow C++ and Python code to communicate with C# code. The C# code will act as the server, handling requests from the C++ and Python clients. I will give you the C# code, so you will need to create a .cpp file for the C++ client and a separate .py file for the Python client.

You will need to research how to use a named pipe client stream on Mac/Linux. Essentially this means opening up two virtual files (one for writing messages to the server and one for reading messages from the server). You then write messages you want to send to the server and read messages you expect to receive as responses.

To help you out, I will give you the code to open both files. Note that the C# server has already created these files when it is executed, so you are just grabbing your own file handle so you can read and write to them.

C++

```
// for messages from client to server
ofstream writer("/tmp/multi-lang-assignment-client2server");

// for messages from server to client
ifstream reader("/tmp/multi-lang-assignment-server2client");
```

Python

```
# for messages from client to server
with open("/tmp/multi-lang-assignment-client2server", 'w') as pipeWriter:
    # code for writing messages you want to send to the server goes here

# for messages from server to client
with open("/tmp/multi-lang-assignment-server2client", 'r') as pipeReader:
    # must be set to non-blocking
    os.set_blocking(pipeReader.fileno(), False)

    # code for reading messages from the server goes here
```

When completing this assignment myself, I found it helpful to open the file every time I wanted to read/write a message, then close the file back. File would have to be reopened for the next message. I also found that including a very small delay in Python between writing a message to the server and reading its response was required. You can use Python's sleep function for this.

After establishing a connection, the first message to send to C# will be the name of the language you are connecting from. This is done by sending a message of the form:

```
name|language\n
```

For example, from C++ it would be:

```
name|C++\n
```

The word "name" goes first, followed by pipe as a delimiter, then the name of the language. The \n puts a newline in the message and is used so that C# knows the line has ended. Don't forget this or else the program will stall, essentially thinking there is more to the message that hasn't arrived yet. This message is essentially a custom format, with pipe as the delimiter. The C# server (as I have written it) is expecting messages of this form.

After sending the name of the language, you will need to invoke functions on the C# side. If you look at the bottom of the C# file, you will notice a class called "Methods". This class contains the methods you will be invoking from the C++ and Python clients. You will pass a message to the C# server in the form of:

```
methodname|argument1|argument2\n
```

For example:

```
add|3|4\n
```

This will invoke the "Add" method sending it arguments of 3 and 4. Same as the name message, a pipe is used as the delimiter. The message you get back will be the returned value of the function (7 in this case). The value sent back to C++ or Python will be a string.

You might want to create several helper methods to help organize your code.

When you end the program, you need to send the following message to the C# server:

```
close\n
```

This will close the connection on the server and reopen a new one so it can wait for a new connection.

Because the server will be executed in the background (more on this below), we need a way to tell the server to exit, instead of opening a new connection and waiting. To do this, we need to send the following message to the server:

```
end\n
```

This will kill the server. Only send this message if the client is ran on the terminal with the command line argument "end" put after it. For example, for C++ this would be:

```
./cpp_client.exe end
```

for Python this would be:

```
python -u py_client.py end
```

More on running the clients can be found below.

One final thing. Because both the server and the connected client will be printing to the standard output at the same time, it would be nice to ensure we know who is speaking. Anytime the C# server talks, it will be in red. When the client talks, there is a special color that should be used. The exact color is shown below.

To get these colors to show in the terminal, is actually really simple. To switch to a new color, just print the following to the terminal:

```
\033[32m
```

The 32 can be replaced with any number from 31 to 37. Each number represents a different color. The exact number to use is also specified below in the output section.

To go back to the default color, just print this:

```
\033[0m
```

This resets the font color. You won't actually see any of this printed, but any messages printed after the color has changed will be printed in that chosen font color. This is called ANSI color codes (feel free to look this up online).

C# Server:

To run the C# server, you will need to first compile the given `cs_server.cs` file. I recommend using Mono for this (<https://www.mono-project.com/download/stable/>). You could also use Homebrew to install Mono. To do that, go to <https://brew.sh> and install the Homebrew software. Then run `brew install mono` in the terminal. After installing Mono, run `csc` in the terminal to compile the server code:

```
csc cs_server.cs
```

This will produce an exe. To run that exe, type the following in the terminal:

```
mono cs_server.exe &
```

This will run the C# server in the background. Notice the ampersand at the end, that tells your shell to run this program behind the scenes and then return normally to the terminal. If it doesn't return to the terminal prompt, just hit enter and it should go back.

If you are having trouble writing the client code, you can put the server into debug mode. This will cause it to print out warning messages and helpful information. Just send it "debug" as a command line argument, i.e.:

```
mono cs_server.exe debug &
```

C++ Client:

After writing the C++ client code, you will need to compile and run it.

On Linux:

To compile type: `g++ -o cpp_client.exe cpp_client.cpp`

To run the client, type: `./cpp_client.exe`

On Mac:

You might have already installed everything needed to compile and run C++, but in case you haven't, I recommend the follow:

- If you haven't already done this, install Homebrew (<https://brew.sh>)
- Then install C/C++ tools: `brew install gcc`
 - The "gcc" package includes tools for both C and C++

Now, with that installed, you can do the following:

To compile type: `g++-14 -o cpp_client.exe cpp_client.cpp`

To run the client, type: `./cpp_client.exe`

When running the C++ client, it should produce the following in the terminal:

```
C#: Connection found
C#: Honoring request from c++ of add(80, 400)... Sending response
C++: receiving response from C#, add(80, 400) = 480
C#: Honoring request from c++ of mul(84, 4)... Sending response
C++: receiving response from C#, mul(84, 4) = 336
C#: Honoring request from c++ of sub(8, 4489)... Sending response
C++: receiving response from C#, sub(8, 4489) = -4481
C#: Honoring request from c++ of div(8, 4)... Sending response
C++: receiving response from C#, div(8, 4) = 2
C#: Connection closed
C#: Waiting for new client connection...
```

The exact arithmetic numbers you send are up to you, but you should send messages for add, mul, sub and div in that order. For the C++ version you should be printing in a green color. This is color code 32 in ANSI colors.

Python Client:

To run the Python client, type: `python -u py_client.py`

Note the "-u". This prevents output buffering ("u" is for "unbuffered"). Without this, all the output will be put into the terminal at the end, making it not match up with the messages from the server.

This should produce the following in the terminal:

```
C#: Connection found
C#: Honoring request from python of add(6, 3)... Sending response
Python: receiving response from C#, add(6, 3) = 9
C#: Honoring request from python of mul(6, 3)... Sending response
Python: receiving response from C#, mul(6, 3) = 18
C#: Honoring request from python of sub(6, 3)... Sending response
Python: receiving response from C#, sub(6, 3) = 3
C#: Honoring request from python of div(6, 3)... Sending response
Python: receiving response from C#, div(6, 3) = 2
C#: Connection closed
C#: Waiting for new client connection...
```

For the Python version you should be printing in a cyan color. This is color code 36 in ANSI colors.

Killing the server:

Once you have command line arguments working on the client, killing the server will look like this:

```
python -u py_client.py end
```

C#: Connection found

C#: Honoring request from python of add(6, 3)... Sending response
Python: receiving response from C#, add(6, 3) = 9

C#: Honoring request from python of mul(6, 3)... Sending response
Python: receiving response from C#, mul(6, 3) = 18

C#: Honoring request from python of sub(6, 3)... Sending response
Python: receiving response from C#, sub(6, 3) = 3

C#: Honoring request from python of div(6, 3)... Sending response
Python: receiving response from C#, div(6, 3) = 2

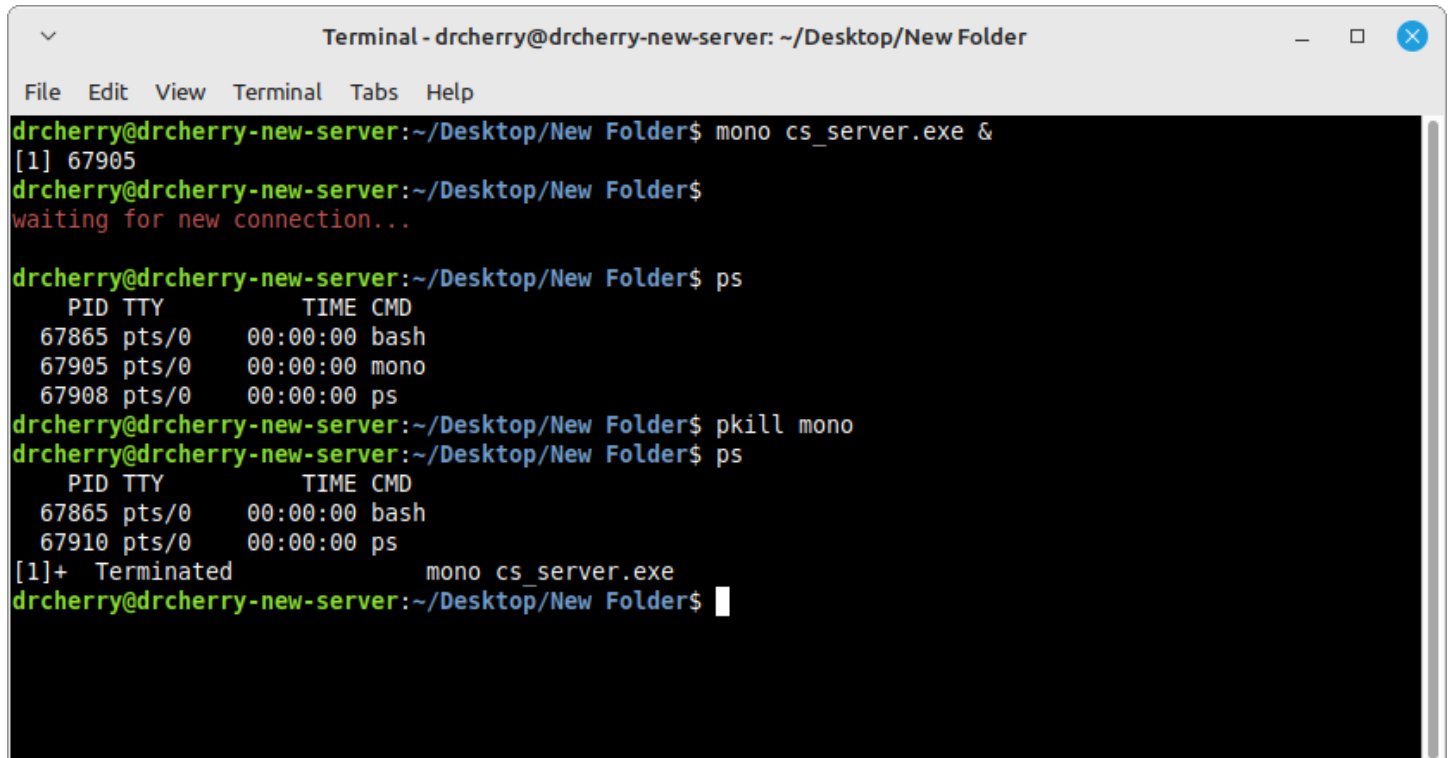
C#: Connection closed

C#: Server closed

```
[1]+  Done                  ./cs_server.exe
```

A similar effect should happen for the C++ client, i.e. the additional "server closed" text will appear at the bottom when you send the "end\n" message.

If you haven't got the command line argument working yet or if you are having trouble stopping the server, you can simply run the "pkill" command from the terminal, giving it mono (since mono is what is running our cs server). You can also use "ps" to see a list of running programs.



```
Terminal - drcherry@drcherry-new-server: ~/Desktop/New Folder
File Edit View Terminal Tabs Help
drcherry@drcherry-new-server:~/Desktop/New Folder$ mono cs_server.exe &
[1] 67905
drcherry@drcherry-new-server:~/Desktop/New Folder$
waiting for new connection...

drcherry@drcherry-new-server:~/Desktop/New Folder$ ps
  PID TTY          TIME CMD
  67865 pts/0    00:00:00 bash
  67905 pts/0    00:00:00 mono
  67908 pts/0    00:00:00 ps
drcherry@drcherry-new-server:~/Desktop/New Folder$ pkill mono
drcherry@drcherry-new-server:~/Desktop/New Folder$ ps
  PID TTY          TIME CMD
  67865 pts/0    00:00:00 bash
  67910 pts/0    00:00:00 ps
[1]+  Terminated                  mono cs_server.exe
drcherry@drcherry-new-server:~/Desktop/New Folder$
```

For submission:

Submit a zip file with both the `cpp_client.cpp` file and the `py_client.py` file in it. Do not submit the C# server file. For grading, I will run the same C# server that I gave you and test both of your clients with this. Submit all files to Canvas.

Rubric:

#	ITEM	POINTS
1	C++ client	20
2	Python client	20
3	Terminal text colors	5
4	Output is correct	5
5	Both clients can send "end" to kill the server	10
	TOTAL	60

Penalties

#	PENALTIES	POINTS
1	Doesn't compile	-50%
2	Doesn't execute once compiled (i.e. it crashes)	-25%
3	Late up to 1 day	-25%
4	Late up to 2 days	-50%
5	Late after 2 days	-100%